

Projet Segmentation en Waterpixels, Philippe Gratias et Maël Mathevet

Ce projet a été encadré par Christophe Kervazo, l'objectif était d'implémenter la méthode de segmentation d'images présentée dans l'article suivant :
Le projet a été réalisé en Python.

La méthode proposée est la suivante :

- 1 - On ne modifie pas l'image si elle est en niveau de gris, sinon on la convertit de l'espace de couleur RGB à l'espace Lab et on applique la méthode sur le canal L
- 2 - Calculer le gradient de l'image à segmenter
- 3 - recouvrir l'image du gradient par une grille régulière hexagonale en appliquant un facteur d'homothétie $0 < \rho < 1$ à chaque hexagone pour créer de l'espace entre chacun d'eux
- 4 - Calculer le minimum du gradient au sein de chaque cellule (=hexagone) et ne garder que la plus grande composante connexe de ce minimum, on appelle ces composantes des marqueurs
- 5 - A partir de cette nouvelle image, calculer une carte de distance aux marqueurs (chaque pixel a pour valeur la distance au marqueur le plus proche)
- 6 - Régulariser spatialement le gradient, c'est-à-dire créer une nouvelle image où chaque pixel (x,y) a pour valeur $grad(x,y) + k * \frac{2}{\sigma} d((x,y), \text{marqueurs})$ où σ est le pas de la grille régulière (où $grad(x,y)$ est la valeur du gradient au pixel (x,y) , $d((x,y), \text{marqueurs})$ est la distance calculée précédemment et k est une constante à fixer pour chaque application, selon qu'on veut donner plus d'importance à la distance aux marqueurs ou au gradient)
- 7 - Enfin on applique l'algorithme du watershed : on considère l'image précédente comme une carte de hauteur et on simule le remplissage de cette carte par de l'eau ce qui nous donne notre segmentation

L'article propose également d'utiliser une grille carrée plutôt qu'hexagonale mais cela génère des waterpixels dont la forme est proche de celle d'un carré, il est difficile de régler une bonne valeur pour le paramètre k .

Il préconise donc de calculer un gradient morphologique, de fixer $\rho = 2/3$, de tester des valeurs de k entre 0 et 16 et d'utiliser une grille hexagonale dont les centres sont espacés d'environ (largeur de l'image)/10

La segmentation en waterpixels est un cas particulier de segmentation en superpixels dont l'intérêt est de proposer un découpage de l'image en régions vérifiant les quatre propriétés suivantes :

- Les niveaux de gris au sein d'une région sont homogènes
- Les superpixels forment une partition de l'image
- Les bords des objets sont inclus dans les bords des superpixels (ils ne les traversent pas)
- Les superpixels forment un motif régulier sur l'image

Les deux dernières conditions sont assez contradictoires, on essaye de trouver un compromis entre les deux avec le choix de la grille et de la valeur de k .

I- Pré traitement :

On commence par convertir l'image en Lab grâce à la méthode `cvtColor` d'openCV si l'image n'est pas déjà en niveaux de gris.

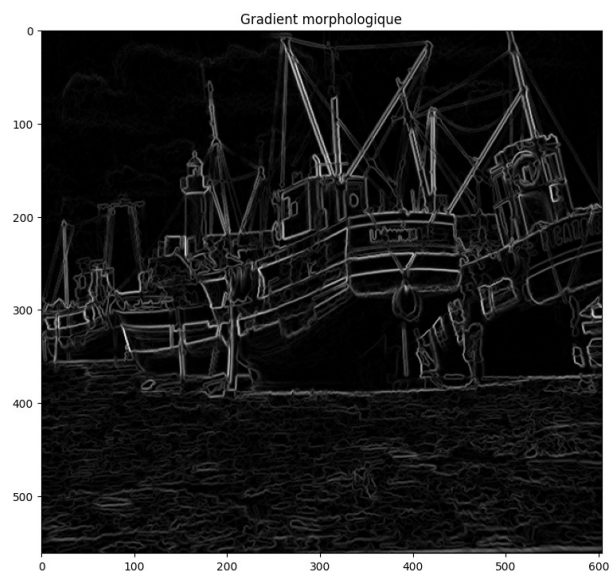
Ensuite on réalise une ouverture puis une fermeture morphologique sur l'image avec comme élément structurant un disque de rayon (écart entre les centres des hexagones)²/16, cette valeur est conseillée dans l'article. L'ouverture va supprimer les zones claires de petite taille et la fermeture les zones sombres, ceci va donc supprimer les petits détails de l'image qui donnent naissance à des waterpixels très petits et donc inintéressants (certains sont mêmes parfois englobés dans d'autres). Le résultat est une version lissée de l'image, elle est plus homogène mais les contours restent toujours visibles et ne sont pas dénaturés.

On passe ainsi de l'image de gauche à celle de droite ci-dessous :



II- Calcul du gradient :

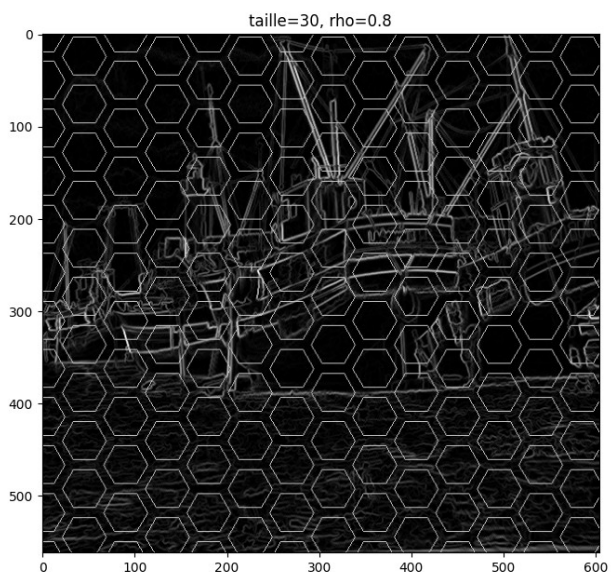
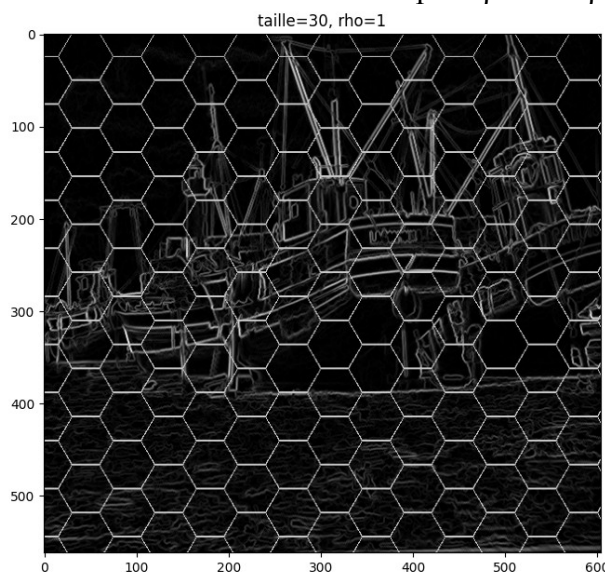
On a choisi, comme le conseille l'article, de calculer un gradient morphologique de l'image (la différence entre la dilatation et l'érosion) avec un disque de rayon 1 comme élément structurant afin d'avoir des contours les plus fins possibles :



III- Création de la grille hexagonale

Pour créer la grille hexagonale, on commence par calculer la position des centres des hexagones grâce à la taille de leurs cotés. Ensuite on applique notre facteur d'homothétie ρ à cette longueur et on calcule les coordonnées des sommets de l'hexagone à partir de celles du centre et ce pour chaque hexagone puis on dessine chaque hexagone sur l'image du gradient de l'image avec le module ImageDraw de la librairie python PIL (Python Image(ing) Library).

On obtient les résultats suivants pour $\rho = 1$ et $\rho = 0,8$.

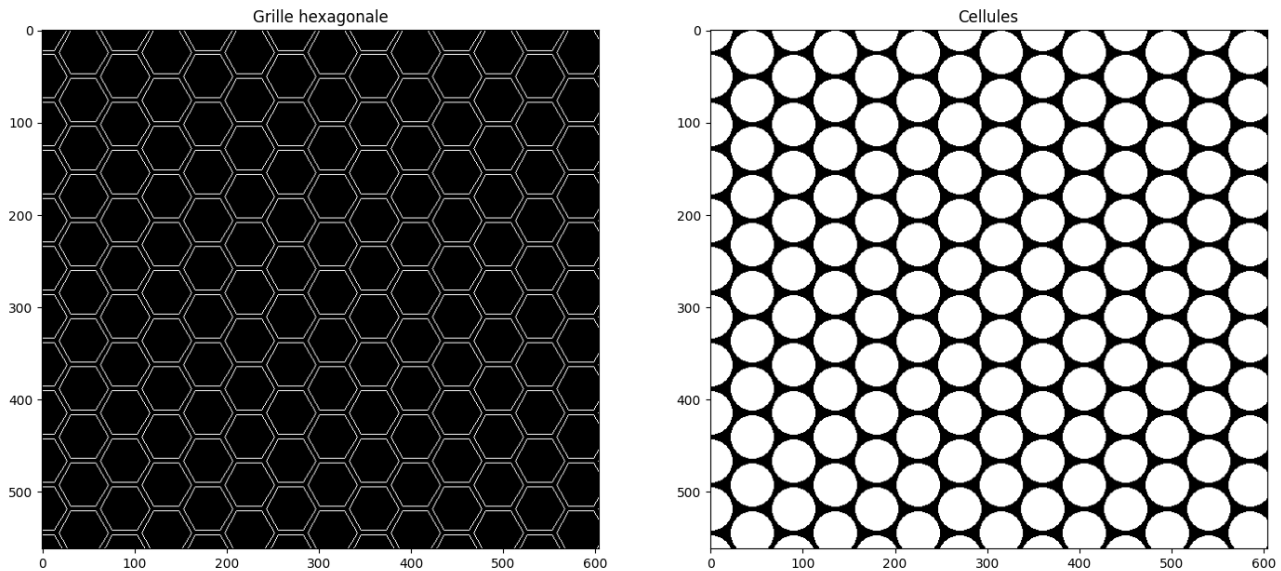


IV- Sélection des marqueurs

1.

Il faut déjà que nous soyons capable de déterminer si un point se trouve ou non dans un hexagone donné. Nous avons dans un premier temps décidé de faire l'approximation suivante : Un point se situe dans un hexagone si et seulement si il se situe dans le cercle inscrit à l'hexagone. Ceci n'est pas rigoureusement exact car on ignore alors quelques points situés près des sommets. Or cela n'est pas vraiment gênant dans notre application car si un marqueur se trouve près du bord d'un hexagone et que cela est également le cas pour les hexagones voisins, il en résulte des waterpixels de plus petite taille car ceux-ci vont rentrer en compétition les uns avec les autres. C'est pour éviter cela que l'on applique un facteur d'homothétie ρ lors de la création de la grille, ainsi les marqueurs sont

garantis d'être un peu éloignés. Pour un premier code, il n'était donc pas problématique de ne pas compter ces points. Cela donnait la carte d'appartenance suivante :



Après avoir mené à bout l'implémentation de l'algorithme de segmentation, nous sommes revenus sur ce point pour compter tous les points de l'hexagone comme appartenant à l'hexagone. Nous n'avons pas eu le temps de le mettre en place mais nous avons testé la méthode suivante :

Pour cela nous avons utilisé une distance de chanfrein appelée la distance_{5,7,11}, cette distance consiste en une approximation de la distance euclidienne dont les boules sont des hexagones :

	11		11	
11	7	5	7	11
	5	0	5	
11	7	5	7	11
	11		11	

Ci dessus le masque permettant de déterminer la distance d'un point au point valant 0. Si on donne des coordonnées à chaque case : (i,j) pour i et j entre 1 et 5 alors on peut déterminer la distance au point (3,3) des cases en blanc grâce à la formule suivante :

Par exemple avec la case (1,1) : $\min(11+(2,3), 11+(3,2), 7+(2,2), \text{etc})$ pour tous les points du masque
Ainsi (1,1) = 14

On trouve de la même manière (1,3) = 10

La carte des distances est donc :

14	11	10	11	14
11	7	5	7	11
10	5	0	5	10
11	7	5	7	11
14	11	10	11	14

Afin de savoir si un point appartient à un hexagone, on calcule la carte de distance de $d_{5,7,11}$, et on choisit un des sommets de l'hexagone. Si un pixel est à une distance euclidienne au centre de

l'hexagone inférieure à celle du sommet (le pixel appartient au cercle circonscrit à l'hexagone) et si sa valeur dans la carte de distance de $d_{5,7,11}$ est inférieure à celle du sommet alors ce pixel appartient à l'hexagone.

Pour calculer la carte de distance, on commence par mettre tous les centres de hexagones à la valeur 0 puis pour chaque pixel on fait passer le masque partiel suivant de haut en bas:

	11	
11	7	5
	5	0
11	7	
	11	

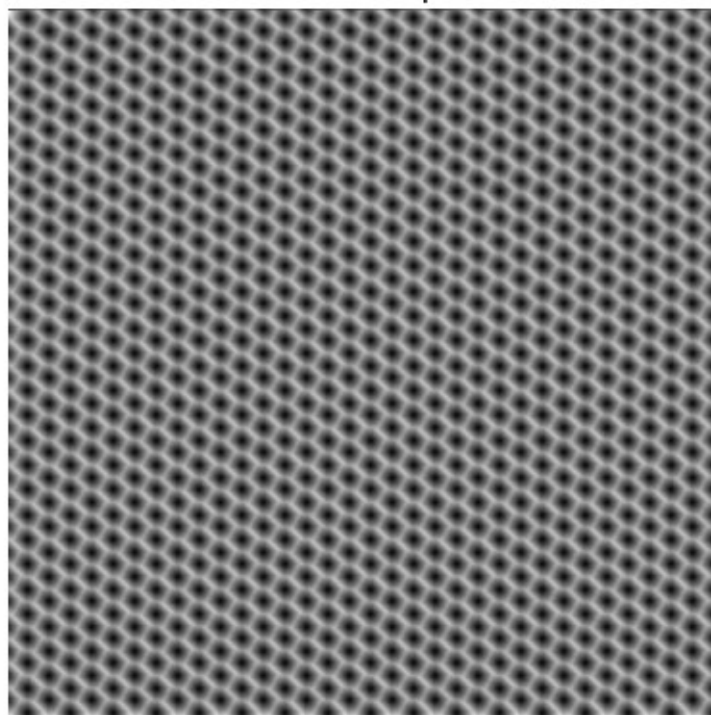
Puis le masque suivant de bas en haut:

	11	
	7	11
0	5	
5	7	11
	11	

Pour calculer la distance entre le point sur lequel le masque est centré et les points à sa droite et sous lui dans un rectangle de taille 5 lignes x 3 colonnes.

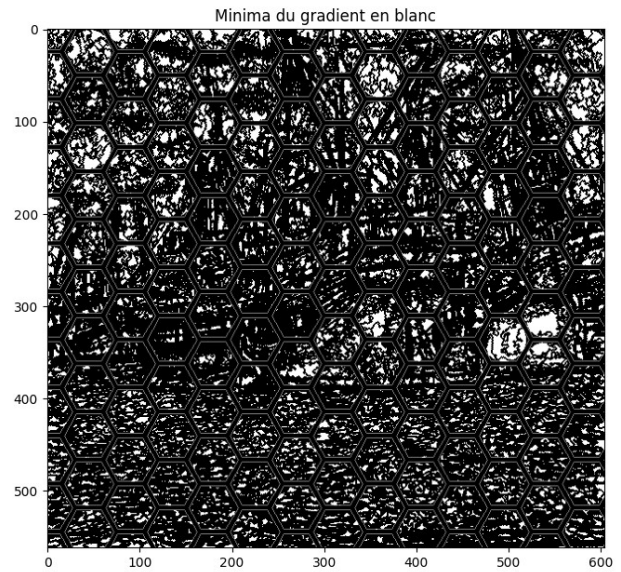
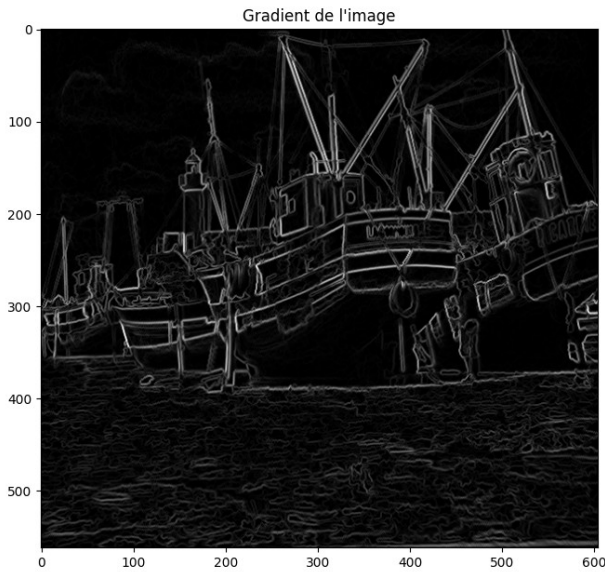
On obtient la carte de distance suivante pour un côté des hexagones égal à 10 :

Carte des distances pour $D_{5,7,11}$



2.

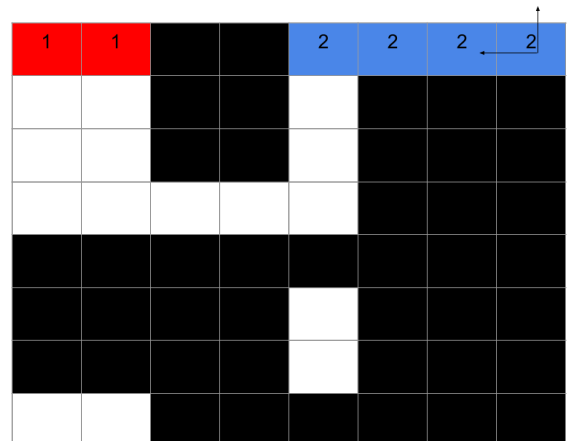
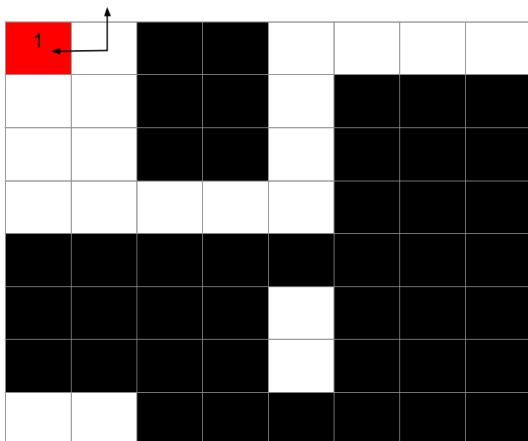
Ensuite on calcule le minimum du gradient au sein de chaque hexagone et on crée une image binaire dans laquelle les points valent 1 si et seulement si le gradient y vaut sa valeur minimale dans l'hexagone auquel il appartient.



Enfin on applique un algorithme de labellisation pour déterminer les composantes connexes du minima du gradient dans chaque hexagone et ne garder que la plus grande.

Cet algorithme est disponible via la méthode `scipy.ndimage.label` mais nous avons décidé de l'implémenter nous-même.

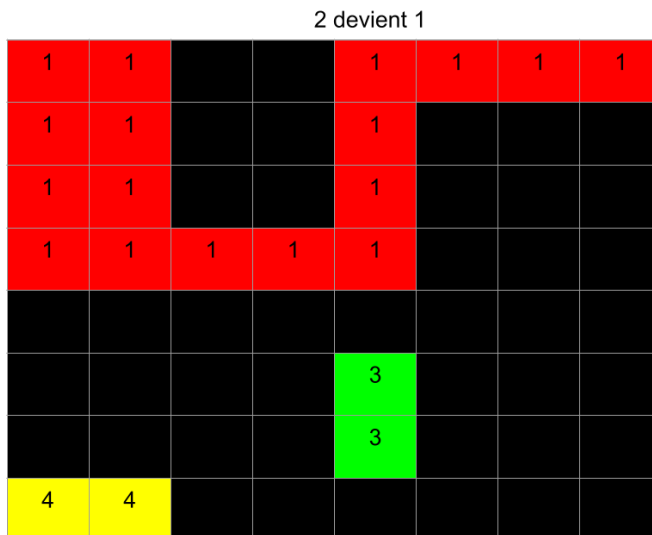
Pour cela on ne s'intéresse qu'à un hexagone à la fois dans l'image obtenue ci-dessus. On parcourt cet hexagone ligne par ligne en affectant des numéros à chaque pixel non nul : tant qu'un pixel possède un de ses voisins de gauche ou du dessus qui a déjà un numéro, on lui donne le même numéro que son voisin :



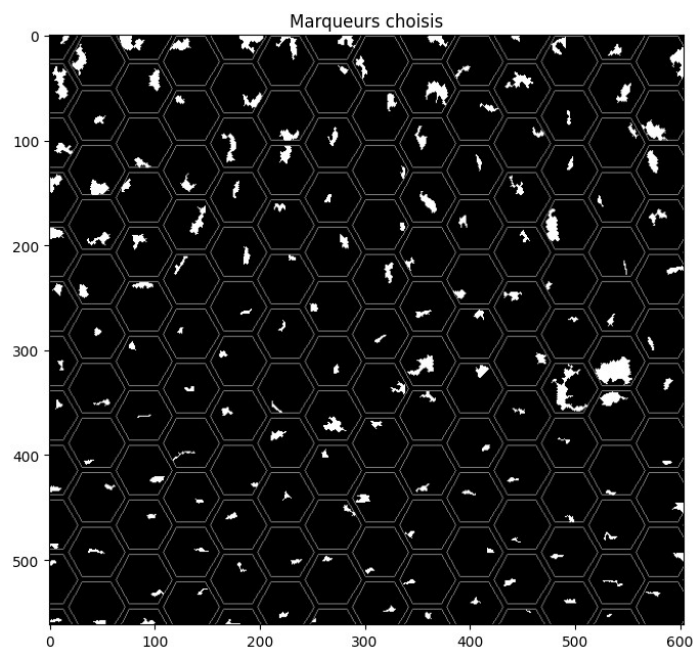
Si les deux voisins ont un numéro différent on choisit la numéro le plus petit et on note le fait que le deuxième numéro doit devenir le premier.



A la fin de l'algorithme, on repassera sur l'image pour remplacer les valeurs qui le nécessitent.

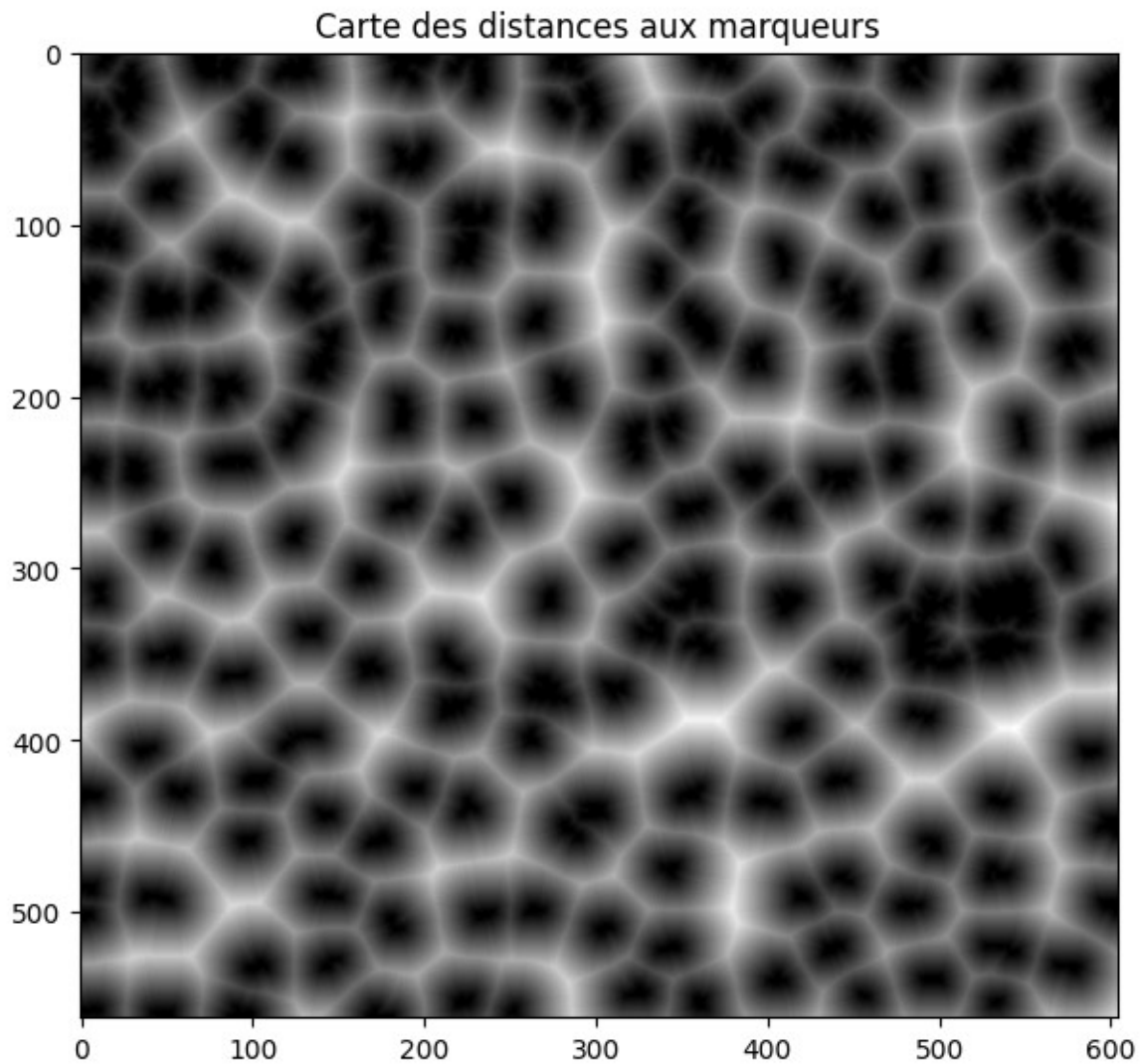


Enfin on compte le nombre de pixel par composante (le nombre de pixel ayant obtenu la même valeur lors de la labellisation) et on ne garde que ceux ayant le numéro le plus représenté.



V- Carte de distance aux marqueurs

Nous avons calculé cette carte grâce à la méthode `distance_transform_edt` de `scipy.ndimage` à partir de l'image des marqueurs, chaque pixel a pour valeur sa distance euclidienne au marqueur le plus proche



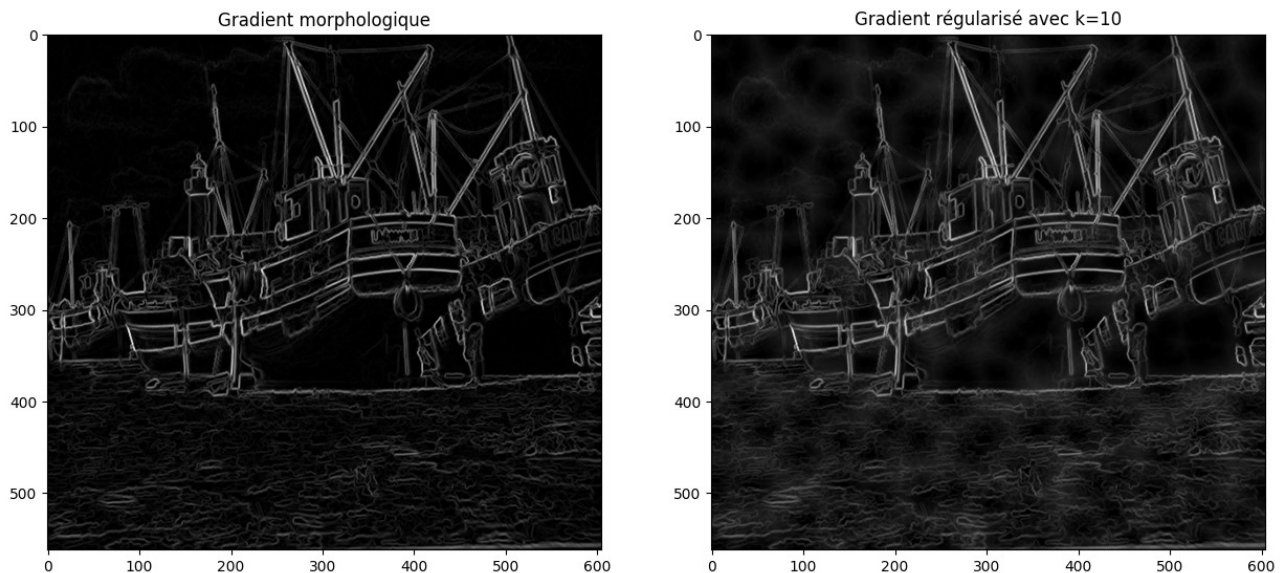
VI- Régularisation spatiale du gradient

On définit un nouveau gradient :

$g_{reg} = g + kd_Q$ où : g est le gradient calculé dans la partie II, k est une constante à choisir et

$d_Q = \frac{2}{\sigma} d_{marqueurs}$ avec σ le pas de la grille hexagonale ($\sqrt{3}$ fois le coté de l'hexagone) et $d_{marqueurs}$ est la distance calculée ci-dessus

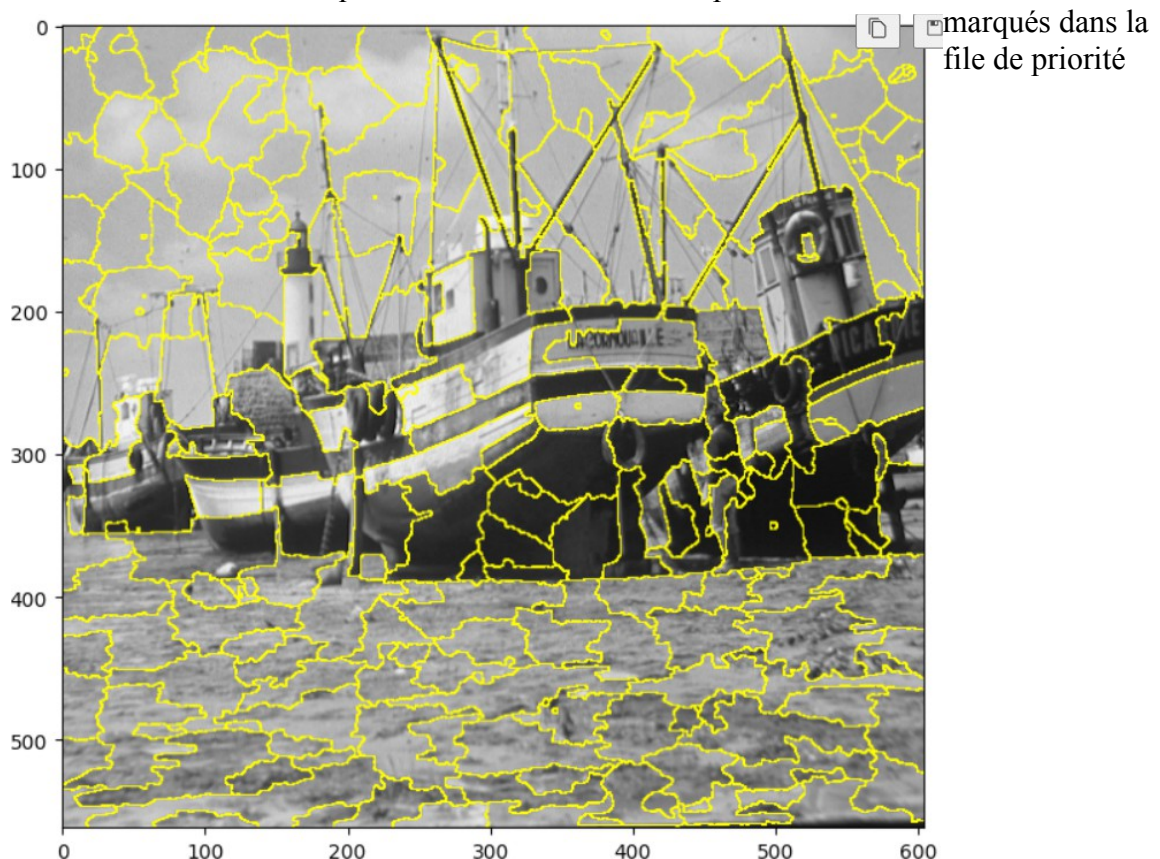
Voici le résultat avec $k = 10$:



VII- Application du watershed

On applique l'algorithme du watershed au gradient régularisé :

- On va générer un waterpixel par marqueur, on commence par marquer ces derniers
- Depuis chaque marqueur, on rajoute les pixels voisins dans une file de priorité dont la priorité vaut leur valeur de gradient régularisé
- On extrait le pixel de plus haute priorité (de plus faible g_{reg}), on le marque, on le rajoute dans un waterpixel si tous ses voisins marqués sont dans le même waterpixel et on met tous ses voisins non



VIII- Tests sur l'influence des paramètres

On peut faire varier les paramètres suivants :

- σ l'écart entre les centres des hexagones ou c la taille des cotés des hexagones
- ρ le facteur d'homothétie pour la génération de la grille
- k le facteur de poids de la distance aux marqueurs dans la régularisation spatiale du gradient

Impact de c :

Résultat pour taille=10



Résultat pour taille=20



Résultat pour taille=30



Résultat pour taille=40



Comme attendu, plus c est grand, moins on obtient de waterpixels.

En effet le nombre de waterpixels obtenu vaut environ $\frac{n}{4c^2}$ avec n le nombre de pixels de l'image

Impact de ρ :

Résultat pour $\rho=1$



Résultat pour $\rho=0.8$



Résultat pour $\rho=0.6$



Résultat pour $\rho=0.4$



Les valeurs de ρ proches de 1 donnent naissance à des waterpixels de très petites tailles englobés dans d'autres waterpixels mais prendre ρ trop petit revient à tronquer fortement les marqueurs, il faut alors réaliser un compromis, la valeur $\frac{2}{3}$ proposée dans l'article semble justifiée au vu des résultat pour $\rho = 0,6$.

Impact de k :

Résultat pour k=1



Résultat pour k=4



Résultat pour k=10



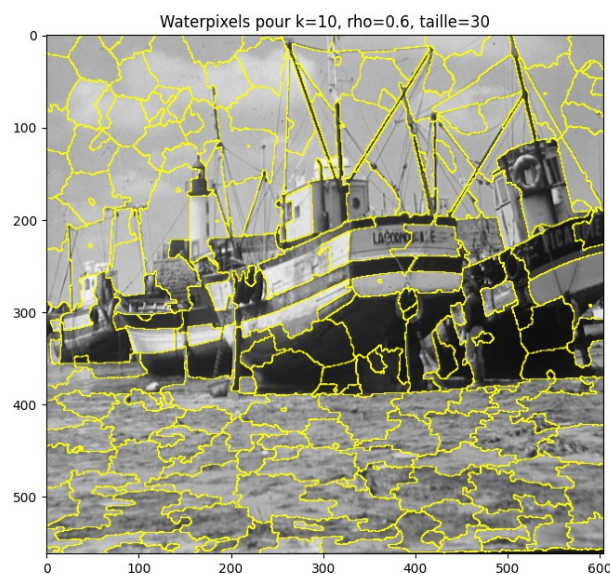
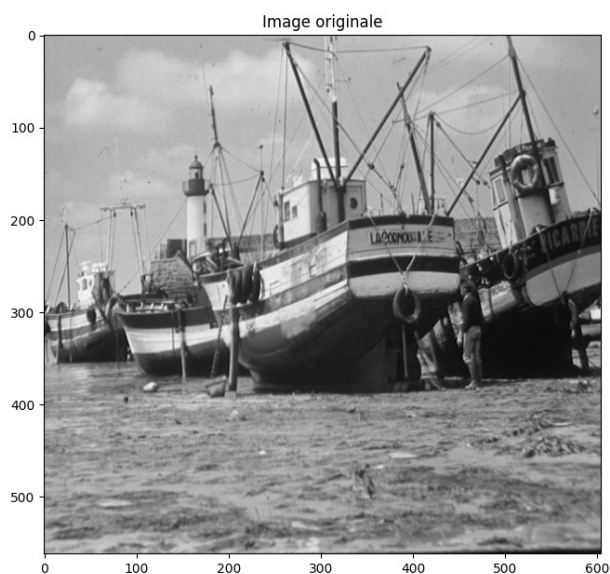
Résultat pour k=20



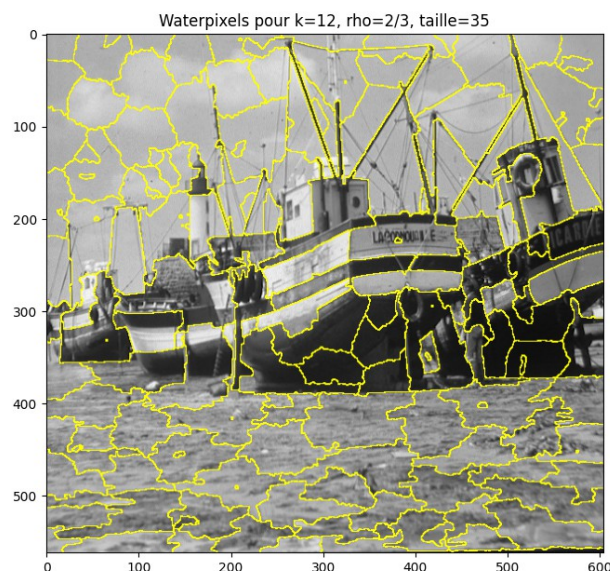
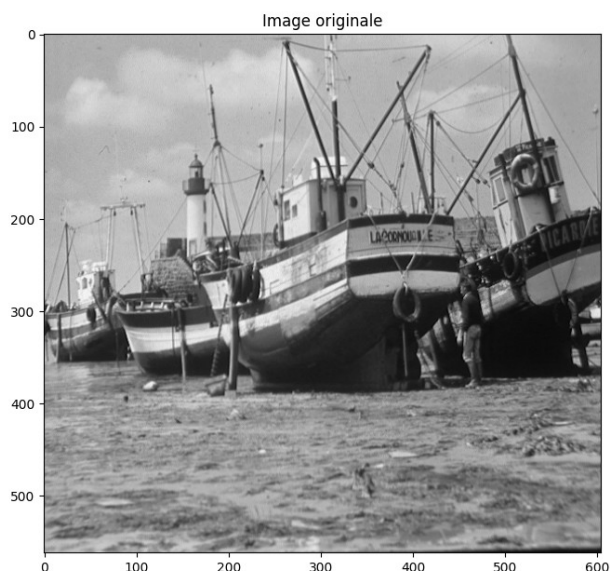
L'article conseille de faire varier k entre 0 et 16, nous avons testé des valeurs entre 1 et 20. Les petites valeurs de k laissent plus d'importance au gradient et permettent l'apparition de petits waterpixels dus à la présence d'un petit marqueur dans un hexagone où le gradient varie brusquement et proche des marqueurs des hexagones voisins où le gradient est plus lisse. Au contraire lorsque k augmente, la segmentation tend vers la tessellation de Voronoï de l'image par les marqueurs. Encore une fois il est nécessaire de choisir une valeur de k moyenne, pour obtenir les avantages des deux extrêmes : avoir des waterpixels plus réguliers en limitant l'apparition de petits waterpixels mais en gardant les contours des objets entièrement dans les waterpixels. En effet dans l'image pour $k=20$ certains contours suivent les câbles des bateaux mais pas sur leur intégralité, ils s'en éloignent à un moment.

On génère donc une dernière segmentation en prenant des valeurs que nous jugeons optimales pour ces trois paramètres et en comparant avec les valeurs choisies dans l'article :

Nos
valeurs



Valeurs
de
l'article



En choisissant de bonnes valeurs d'après nos observations précédentes, on a pu régler la majorité des problèmes observés : les waterpixels suivent bien les bords des objets jusqu'au bout et il y a peu voire pas de très petits pixels.

Néanmoins un problème reste présent, qui n'apparaît pas dans les exemples de l'article, de petits contours se trouvent à l'intérieur de certains waterpixels et ne devraient pas exister car ils ne correspondent pas à un marqueur. Nous ne savons pas quelle est leur origine mais il serait possible de les supprimer en enlevant les waterpixels de trop petite taille après l'application de l'algorithme.